

HireAI Copilot

AI-Powered Recruitment Intelligence Platform

Machine Learning Engineers

Scoring Model, Evaluation & ML API — 4 Interns

Confidential · Version 1.0 · 6-Week Intern Program

This document is the complete build guide for the Machine Learning team. It covers everything four junior interns need to build, evaluate, and deploy the candidate scoring model that powers HireAI Copilot's ranking engine over a 6-week sprint: feature engineering, training XGBoost and Random Forest models, SHAP explainability, and the production ML API.

IMPORTANT

A perfect model that nobody can call is worth zero. A 'good enough' model behind a working API is worth everything. Always have a deployable version of whatever you have, even if it's a one-line baseline.

Table of Contents

1. Team Overview & Scope
2. Tech Stack & Project Setup
3. Individual Intern Allocations
4. The Scoring Problem — Math & Definitions
5. Feature Engineering
6. Model Training
7. Model Evaluation
8. SHAP Explainability
9. ML API & Deployment
10. 6-Week Roadmap with Deadlines
11. Cross-Team Integration
12. Definition of Done

01 Team Overview & Scope

1.1 Who We Are

Four junior interns, no team lead. We split work along the standard ML lifecycle: M1 cleans data, M2 trains models, M3 evaluates and explains, M4 serves the model in production. Daily standup at 10:00 AM with the PM and other teams; internal ML sync (15 min) at 4:00 PM.

1.2 What We Own

Owner	Deliverable
M1	Clean, feature-engineered dataset ready for training. Train/val/test splits.
M2	Trained XGBoost regressor (predicts score 0-100) and Random Forest classifier (predicts label).
M3	Evaluation metrics, SHAP feature importance, formal Model Evaluation Report.
M4	FastAPI service exposing /score endpoint, with model versioning and a rule-based fallback.

1.3 What We Do NOT Build

- We do not parse resumes — GenAI's G1 owns that. We consume parsed candidates from the DB.
- We do not write explanations in natural language — GenAI's G3 owns that. We provide SHAP values and the structured score breakdown; G3 turns it into prose.
- We do not load CSVs into PostgreSQL — Data Analysts own that. We read scores.csv via a query.
- We do not build dashboards — Web team owns visualization.

NOTE

Our scoring data (scores.csv) is synthetic and pre-computed. We are training a model to mimic that ground-truth scoring, not to invent it. The interesting questions are: (1) Can we predict score within 5 MAE? (2) Can we classify Good/Average/Poor Fit at >88% accuracy? (3) Are our predictions explainable?

02

Tech Stack & Project Setup

2.1 Stack

Tool	Why & How
Python 3.11	Runtime.
pandas + numpy	Data manipulation. M1 lives in pandas all day.
scikit-learn 1.3+	RandomForest classifier, train/test split, preprocessing utilities.
xgboost 2.0+	XGBoost regressor for score prediction.
SHAP	Feature importance and per-prediction explainability.
matplotlib + seaborn	EDA and evaluation plots.
jupyter notebook	EDA only. No production code in notebooks — convert everything to .py modules before merge.
joblib	Saving/loading trained models (.pkl).
FastAPI	M4 serves the model via REST.
pytest	Unit + regression tests for the API.
mlflow (optional)	Experiment tracking. Skip if it slows you down — a CSV log of (run_id, params, metrics) is fine for a 6-week project.

2.2 Repository Layout

```

hireai-ml/
├─ notebooks/                # EDA only
│  └─ 01_eda.ipynb           # M1 owns
│  └─ 02_baseline_rule.ipynb # M2 owns – sanity check before ML
│  └─ 03_feature_imp.ipynb   # M3 owns – SHAP exploration
│  └─ 04_error_analysis.ipynb # M3 owns – where does the model fail?
├─ src/
│  └─ data/
│     └─ load.py              # M1
│     └─ clean.py             # M1
│     └─ features.py          # M1 – feature engineering
└─ models/

```

```

| | | └─ train_regressor.py # M2
| | | └─ train_classifier.py # M2
| | | └─ tune.py           # M2 – hyperparameter search
| | | └─ baseline.py       # M2 – rule-based fallback
| | └─ eval/
| | | └─ metrics.py        # M3
| | | └─ shap_analysis.py  # M3
| | | └─ report.py         # M3 – generates PDF report
| | └─ api/
| | | └─ main.py           # M4 – FastAPI app
| | | └─ schemas.py        # M4
| | | └─ service.py        # M4 – prediction logic
| | └─ config.py           # paths, model versions, thresholds
└─ data/
    └─ raw/                # CSVs from Data Analyst team (read-only)
    └─ processed/          # M1 writes here
    └─ splits/             # M1 writes here (train.csv, val.csv, test.csv)
└─ models/                 # M2 writes here (.pkl files, versioned)
    └─ regressor_v1.pkl
    └─ classifier_v1.pkl
    └─ manifest.json       # which version is "current"
└─ reports/
    └─ eval_v1.pdf         # M3 outputs
    └─ shap/
└─ tests/
    └─ test_features.py
    └─ test_api.py
    └─ test_baseline.py
└─ requirements.txt
└─ Dockerfile

```

2.3 Day-1 Setup

```

# 1. Clone
git clone https://github.com/<org>/hireai-ml.git
cd hireai-ml

# 2. Virtualenv
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt

# 3. Get the CSVs from the Data Analyst team
# (DA1 will share the cleaned CSVs in a shared S3 bucket / Drive folder)
cp /path/to/shared/{candidates,jobs,applications,scores}.csv data/raw/

# 4. Verify data loads

```

```
python -c "from src.data.load import load_all; print(load_all().keys())"
```

```
# 5. Run baseline
```

```
python src/models/baseline.py
```

```
# 6. Run API (after M4 is set up)
```

```
uvicorn src.api.main:app --reload --port 8002
```

03

Individual Intern Allocations

M1 — Data Preprocessing & Feature Engineering

Focus: Turn raw CSVs into a clean, feature-rich dataset that M2 can train on. You are the foundation — if your features are weak, no amount of model tuning can save the project.

Core Tasks:

- Load all 4 CSVs (candidates, jobs, applications, scores) into pandas DataFrames via `src/data/load.py`.
- Run a comprehensive EDA in `01_eda.ipynb`: distributions, missing values, outliers, correlations between features and the target score.
- Validate data integrity: every (`candidate_id`, `job_id`) in `scores.csv` should exist in `applications.csv`. Flag any orphans to DA team.
- Engineer features (full list in Section 5): `skill_overlap_count`, `skill_overlap_pct`, `experience_gap`, `project_count`, `education_match`, etc.
- Encode categoricals: department (one-hot), education degree (ordinal: high school < bachelor < master < phd), label (target — leave as-is until M2 maps to int).
- Handle missing values explicitly: never silently impute. Document strategy: median for numeric, 'unknown' for categorical, drop if >30% missing.
- Split data: 70/15/15 stratified by label so train/val/test all have similar Good/Average/Poor proportions. Save as `data/splits/{train,val,test}.csv`.
- Write a 'feature card' for each engineered feature: name, formula, range, why it should be predictive.

Weekly Milestones:

1. Week 1: EDA notebook complete. Data quality report. All 4 CSVs loaded and joined cleanly.
2. Week 2: First version of feature matrix complete (10+ features). Train/val/test split saved. Documented in feature card.
3. Week 3: Feature engineering iteration #2 based on M3's importance analysis. Add/remove features as needed.
4. Week 4: Final feature set frozen. Documentation polished. Splits regenerated for reproducibility.
5. Week 5: Sanity-check that production data (from web team's DB) goes through the same pipeline as training data.
6. Week 6: Final feature engineering report (1-page) bundled with M3's eval report.

M2 — Model Training & Hyperparameter Tuning

Focus: Train two models — a regressor (predicts score 0-100) and a classifier (predicts Good/Average/Poor Fit) — on the features M1 produces. Make them as good as the data allows.

Core Tasks:

- Build a rule-based baseline first (src/models/baseline.py): $\text{composite} = 0.45 * \text{skills_match} + 0.30 * \text{exp_score} + 0.25 * \text{project_score}$. This is your floor — ML must beat it or we don't ship ML.
- Train an XGBoost regressor on the train set with default hyperparameters. Evaluate on val. Report MAE, RMSE, R^2 .
- Train a Random Forest classifier on the train set. Evaluate on val. Report accuracy, F1 (macro).
- Run hyperparameter tuning with GridSearchCV or Optuna: max_depth, learning_rate, n_estimators, min_child_weight (for XGBoost); n_estimators, max_depth, min_samples_split (for RF).
- Keep tuning to a fixed compute budget (e.g. 100 trials). Don't tune forever — diminishing returns kick in fast.
- Save final models to models/ with version suffix: regressor_v1.pkl, classifier_v1.pkl. Update manifest.json with which is current.
- Document training: random seeds, hyperparameters, training time, dataset size, target metric — everything M3 needs to reproduce.
- Coordinate with M3 to make sure test set is held out and never touched during training/tuning.

Weekly Milestones:

7. Week 1: Rule-based baseline implemented. Numbers documented. EDA learnings absorbed.
8. Week 2: XGBoost regressor and RF classifier trained on M1's features. Default hyperparameters. Validation metrics logged.
9. Week 3: Hyperparameter tuning complete. Best models saved as v1. Validation metrics on tuned models documented.
10. Week 4: Re-train with M1's updated features (iteration #2). Bump to v2 if metrics improve.
11. Week 5: Models frozen. Training notebooks cleaned and committed. Reproducibility test: run from scratch, same seeds, same metrics.
12. Week 6: Models v_final saved. Training documentation finalized.

M3 — Model Evaluation & SHAP Explainability

Focus: Measure how good the models are. Find where they fail. Generate SHAP values that GenAI's G3 turns into human-readable explanations.

Core Tasks:

- Compute regression metrics on test set: MAE (target <5), RMSE (target <8), R^2 (target >0.85).
- Compute classification metrics on test set: accuracy (target >88%), precision/recall/F1 per class, confusion matrix.
- Run error analysis in 04_error_analysis.ipynb: where does the model fail most? (e.g. high-skill candidates with low experience). Document patterns.
- Compute SHAP values for the regressor on the entire test set. Generate global feature importance plot (bar chart) and a beeswarm plot.
- For each (candidate_id, job_id) prediction, compute per-prediction SHAP values. Save them in a way M4's API can serve.
- Compare ML model vs rule-based baseline: how much does ML actually win? Quantify the lift.
- Define and write the formal Model Evaluation Report (PDF, 5-8 pages): problem statement, data, methodology, metrics, error analysis, SHAP findings, recommendations. Use src/eval/report.py to generate.
- Coordinate with G3 (GenAI XAI lead) on the SHAP value format — they need feature_name + contribution + direction.

Weekly Milestones:

13. Week 1: Eval framework skeleton in place. Metrics module written (no models to evaluate yet).
14. Week 2: First baseline + first ML model evaluated on val set. Numbers shared with team in standup.
15. Week 3: SHAP analysis on tuned models. Top-10 feature importance plot generated.
16. Week 4: Per-prediction SHAP integration agreed with G3. JSON format finalized.
17. Week 5: Error analysis notebook complete. Test-set metrics on final models documented.
18. Week 6: Formal Model Evaluation Report (PDF, 5-8 pages) delivered to PM. Demo materials prepared.

M4 — ML API & Deployment

Focus: Wrap the trained models in a FastAPI service that the web team's backend can call. Make it fast, reliable, versioned, and gracefully degrades to the rule-based baseline if anything is broken.

Core Tasks:

- Build src/api/main.py: FastAPI app exposing POST /score and POST /score/batch.
- Define request/response Pydantic schemas matching the API contract in Section 9.
- Load the current model from models/manifest.json on startup. Re-load on SIGHUP if you have time.
- On every prediction request: fetch candidate + job from web backend's API (or accept embedded data in the request body), apply M1's feature pipeline, run model.predict(), return result.

- Implement the rule-based fallback: if the ML model fails to load or returns a malformed prediction, fall back to baseline scoring. Always return SOMETHING — never error out.
- Implement /score/batch for bulk inference: useful for pre-computing all (candidate, job) scores when a job is created.
- Add /healthz, /readyz, and /version endpoints. /version returns currently loaded model version.
- Containerize with Dockerfile. Deploy alongside web backend on the demo VM (W5 helps).
- Write integration tests using pytest + httpx. Test the full prediction path with realistic input.

Weekly Milestones:

19. Week 1: FastAPI scaffold. /healthz and /version work. Stub /score returns hardcoded prediction.
 20. Week 2: /score live, calling the rule-based baseline (no ML yet). Web team can integrate.
 21. Week 3: ML model loaded; /score returns real predictions. Fallback logic tested by simulating a model load failure.
 22. Week 4: /score/batch live. Used by GenAI G3 to pre-cache explanations.
 23. Week 5: Containerized. Deployed to demo VM. Integration tests passing.
 24. Week 6: Full E2E demo flow validated. Latency p95 <300ms documented.
-

04

The Scoring Problem — Math & Definitions

4.1 What We Are Predicting

For each (candidate, job) pair, predict TWO things:

Output	Type	Description
score	Regression	Continuous 0-100 composite score
label	Classification	Categorical: 'Good Fit' / 'Average Fit' / 'Poor Fit'
skills_match	Regression	0-100, % of required skills the candidate has
exp_score	Regression	0-100, how well experience matches
project_score	Regression	0-100, project portfolio relevance

In practice, `skills_match`, `exp_score`, and `project_score` are computed deterministically from features (not predicted). The regressor predicts the OVERALL score; the classifier predicts the LABEL. Both are trained on `scores.csv` as ground truth.

4.2 Targets

Metric	Acceptable	Excellent
Regressor MAE	< 8	< 5
Regressor RMSE	< 12	< 8
Regressor R ²	> 0.75	> 0.85
Classifier accuracy	> 80%	> 88%
Classifier macro-F1	> 0.75	> 0.85
ML lift over baseline	> 5 MAE points	> 10 MAE points

4.3 How Scores.csv Was Generated

NOTE

`scores.csv` is synthetic — generated by Data Analysts via a known formula plus added noise. This

means a perfect model would essentially recover the formula. We aim for 'very good', not 'perfect'
— overfitting to synthetic noise is a real risk.

05

Feature Engineering

5.1 Required Features (M1 must build all of these)

Feature	Type	Computation
skill_overlap_count	int	$\text{len}(\text{set}(\text{candidate.skills}) \cap \text{set}(\text{job.required_skills}))$
skill_overlap_pct	float	$\text{skill_overlap_count} / \text{len}(\text{job.required_skills})$
candidate_skill_count	int	$\text{len}(\text{candidate.skills})$
job_skill_count	int	$\text{len}(\text{job.required_skills})$
experience_years	float	candidate.experience_years
min_experience	float	job.min_experience
experience_gap	float	candidate.experience_years - job.min_experience
experience_meets	bool	candidate.experience_years $\geq$ job.min_experience
project_count	int	$\text{len}(\text{candidate.projects})$
education_level	ordinal	0=high school, 1=bachelor, 2=master, 3=phd
department_*	one-hot	Engineering, Data Science, Design, DevOps, etc.
semantic_skill_overlap	float	Cosine similarity of skills sentence embeddings (optional)

5.2 Feature Card Template

For each feature, M1 maintains a 'feature card' explaining what it is and why it matters. Example:

```
# Feature: experience_gap
- Type: float
- Range: [-8, +20] (negative if under-qualified, positive if over-qualified)
- Formula: candidate.experience_years - job.min_experience
- Why predictive: roles need a minimum experience; both under- and over-experience may hurt fit.
- Missing handling: if min_experience is null, use 0 (assume entry-level).
- Distribution: skewed right; median ~+1.5 yrs, p95 ~+8 yrs.
- Correlation with score: 0.42 (moderate positive).
```


06

Model Training

6.1 Baseline (Run This First)

```
# src/models/baseline.py
def predict_baseline(features: dict) -> tuple[float, str]:
    skills_match = 100 * features["skill_overlap_pct"]
    exp_score = 100 * min(features["experience_years"] /
max(features["min_experience"], 1), 1.0)
    project_score = min(20 * features["project_count"], 100)

    score = 0.45 * skills_match + 0.30 * exp_score + 0.25 * project_score
    if score >= 75: label = "Good Fit"
    elif score >= 50: label = "Average Fit"
    else: label = "Poor Fit"
    return score, label
```

6.2 XGBoost Regressor

```
# src/models/train_regressor.py
import xgboost as xgb
import joblib, json
from src.data.load import load_splits

X_train, y_train, X_val, y_val, X_test, y_test = load_splits()

model = xgb.XGBRegressor(
    n_estimators=300, max_depth=6, learning_rate=0.05,
    subsample=0.8, colsample_bytree=0.8,
    objective="reg:squarederror", random_state=42,
    early_stopping_rounds=20,
)
model.fit(X_train, y_train, eval_set=[(X_val, y_val)], verbose=False)

joblib.dump(model, "models/regressor_v1.pkl")
manifest = {"current_regressor": "regressor_v1.pkl",
            "current_classifier": "classifier_v1.pkl"}
json.dump(manifest, open("models/manifest.json", "w"), indent=2)
```

6.3 Hyperparameter Tuning

Use GridSearchCV with a small grid. Don't overdo it — 50-100 trials max.

```
from sklearn.model_selection import GridSearchCV

param_grid = {
    "n_estimators": [200, 300, 500],
    "max_depth":    [4, 6, 8],
    "learning_rate": [0.03, 0.05, 0.1],
}
gs = GridSearchCV(xgb.XGBRegressor(random_state=42),
                  param_grid, cv=3, scoring="neg_mean_absolute_error", n_jobs=-1)
gs.fit(X_train, y_train)
print("Best:", gs.best_params_, "MAE:", -gs.best_score_)
```


07

Model Evaluation

7.1 Required Metrics

```
# src/eval/metrics.py
from sklearn.metrics import (
    mean_absolute_error, mean_squared_error, r2_score,
    accuracy_score, f1_score, precision_recall_fscore_support,
    confusion_matrix, classification_report,
)
import numpy as np

def regression_metrics(y_true, y_pred):
    return {
        "mae": mean_absolute_error(y_true, y_pred),
        "rmse": np.sqrt(mean_squared_error(y_true, y_pred)),
        "r2": r2_score(y_true, y_pred),
    }

def classification_metrics(y_true, y_pred):
    return {
        "accuracy": accuracy_score(y_true, y_pred),
        "f1_macro": f1_score(y_true, y_pred, average="macro"),
        "report": classification_report(y_true, y_pred, output_dict=True),
        "confusion": confusion_matrix(y_true, y_pred).tolist(),
    }
```

7.2 Error Analysis Patterns to Look For

- High-skill, low-experience candidates: do they get penalized too much?
- Over-experienced candidates (e.g. 10y for a 2y job): are they wrongly marked 'Good Fit'?
- Niche skills (e.g. only 1 candidate has it): does the model handle small-sample features sensibly?
- Mislabelled boundary cases (score 73 = 'Average', score 76 = 'Good'): are most errors near boundaries?

7.3 The Eval Report (M3 deliverable)

PDF, 5-8 pages, generated via src/eval/report.py. Sections:

25. Executive summary (1 paragraph): how well does the model work?
26. Problem & dataset: what we predicted, train/val/test sizes.
27. Methodology: model choices, hyperparameters, training time.

28. Metrics table: regression + classification, vs baseline.
29. Confusion matrix and per-class precision/recall.
30. SHAP feature importance: top 10 features.
31. Error analysis: where does the model fail and why.
32. Recommendations: what we'd improve given more time.

08

SHAP Explainability

SHAP (SHapley Additive exPlanations) tells us how much each feature contributed to a specific prediction. This is what powers GenAI's explanation layer.

8.1 Global SHAP (across the test set)

```
# src/eval/shap_analysis.py
import shap, joblib, matplotlib.pyplot as plt
from src.data.load import load_splits

model = joblib.load("models/regressor_v1.pkl")
_, _, _, X_test, _ = load_splits()

explainer = shap.TreeExplainer(model)
shap_values = explainer.shap_values(X_test)

# Global bar plot – feature importance averaged over all predictions
shap.summary_plot(shap_values, X_test, plot_type="bar", show=False)
plt.savefig("reports/shap/global_importance.png", dpi=150, bbox_inches="tight")

# Beeswarm – distribution of SHAP per feature
shap.summary_plot(shap_values, X_test, show=False)
plt.savefig("reports/shap/beeswarm.png", dpi=150, bbox_inches="tight")
```

8.2 Per-Prediction SHAP (for /api/explain)

M4's /score endpoint will return SHAP values for each prediction. M3 defines the format; G3 (GenAI XAI) consumes it.

```
def get_shap_for_prediction(model, x: pd.DataFrame) -> list[tuple[str, float]]:
    explainer = shap.TreeExplainer(model)
    shap_vals = explainer.shap_values(x)[0] # one row
    feature_names = x.columns.tolist()
    pairs = list(zip(feature_names, shap_vals.tolist()))
    # Return top 5 by absolute contribution
    pairs.sort(key=lambda p: abs(p[1]), reverse=True)
    return pairs[:5]
```

IMPORTANT

Computing SHAP per request adds ~50-100ms latency. If that's too much for the demo, pre-

compute SHAP for all (candidate, job) pairs in M4's batch endpoint and cache the results.

09

ML API & Deployment

9.1 API Contract

Endpoint	Method	Description
/score	POST	Body: {candidate_id, job_id} OR {candidate, job} (full objects). Returns ScorePrediction.
/score/batch	POST	Body: {pairs: [{candidate_id, job_id}, ...]}. Returns list of ScorePrediction.
/healthz	GET	Liveness.
/readyz	GET	Readiness — checks model loaded.
/version	GET	Returns currently loaded model version.

9.2 Request / Response Schemas

```
# src/api/schemas.py
from pydantic import BaseModel, Field
from typing import List, Tuple, Literal

class ScoreRequest(BaseModel):
    candidate_id: str
    job_id: str

class ScorePrediction(BaseModel):
    candidate_id: str
    job_id: str
    score: float = Field(..., ge=0, le=100)
    skills_match: float = Field(..., ge=0, le=100)
    experience_score: float = Field(..., ge=0, le=100)
    project_score: float = Field(..., ge=0, le=100)
    label: Literal["Good Fit", "Average Fit", "Poor Fit"]
    shap_top5: List[Tuple[str, float]]
    model_version: str
    used_fallback: bool # True if rule-based fallback triggered
```

9.3 Service Logic with Fallback

```
# src/api/service.py
import joblib, json, logging
from src.models.baseline import predict_baseline
from src.eval.shap_analysis import get_shap_for_prediction

log = logging.getLogger("ml-api")

def load_current_model():
    m = json.load(open("models/manifest.json"))
    return joblib.load(f"models/{m['current_regressor']}"), m['current_regressor']

MODEL, MODEL_VERSION = load_current_model()

def predict(features: dict, candidate_id: str, job_id: str):
    try:
        x = build_feature_row(features) # M1's pipeline
        score = float(MODEL.predict(x)[0])
        label = score_to_label(score)
        shap_top5 = get_shap_for_prediction(MODEL, x)
        return {
            "candidate_id": candidate_id, "job_id": job_id,
            "score": score, "label": label,
            "skills_match": 100 * features["skill_overlap_pct"],
            "experience_score": features.get("exp_score", 0),
            "project_score": features.get("project_score", 0),
            "shap_top5": shap_top5,
            "model_version": MODEL_VERSION,
            "used_fallback": False,
        }
    except Exception as e:
        log.error("ML prediction failed, falling back: %s", e)
        score, label = predict_baseline(features)
        return {
            "candidate_id": candidate_id, "job_id": job_id,
            "score": score, "label": label,
            "skills_match": 100 * features["skill_overlap_pct"],
            "experience_score": 0, "project_score": 0,
            "shap_top5": [],
            "model_version": "rule_based_v1",
            "used_fallback": True,
        }
```

9.4 Performance Targets

Metric	Target	Notes
p50 latency	< 100ms	Single prediction, warm cache.
p95 latency	< 300ms	Including SHAP computation.
Batch throughput	> 200 preds/sec	On /score/batch with batches of 100.
Memory	< 500MB	Model + features pipeline.

10

6-Week Roadmap with Deadlines

Week 1 — Foundations

Inter n	Friday EOD Deliverable
M1	EDA notebook complete. Data quality report. CSVs loaded and joined cleanly.
M2	Rule-based baseline implemented, numbers documented.
M3	Eval framework skeleton. Metrics module written.
M4	FastAPI scaffold. /healthz and /version work. Stub /score returns hardcoded prediction.

Week 2 — First Models

Inter n	Friday EOD Deliverable
M1	First version of feature matrix complete (10+ features). Train/val/test split saved.
M2	XGBoost regressor + RF classifier trained on M1's features. Default hyperparameters.
M3	First baseline + first ML model evaluated on val set. Numbers shared in standup.
M4	/score live, calling rule-based baseline (no ML yet). Web team can integrate.

Week 3 — Tuning & SHAP

Inter n	Friday EOD Deliverable
M1	Feature engineering iteration #2 based on M3's importance analysis.
M2	Hyperparameter tuning complete. Best models saved as v1.
M3	SHAP analysis on tuned models. Top-10 feature importance plot generated.
M4	ML model loaded; /score returns real predictions. Fallback logic tested.

Week 4 — Integration

Inter n	Friday EOD Deliverable
M1	Final feature set frozen. Documentation polished.
M2	Re-train with M1's updated features. Bump to v2 if metrics improve.
M3	Per-prediction SHAP integration agreed with G3. JSON format finalized.
M4	/score/batch live. Used by GenAI G3 to pre-cache explanations.

Week 5 — Polish

Inter n	Friday EOD Deliverable
M1	Sanity-check that production data goes through same pipeline as training data.
M2	Models frozen. Reproducibility test passes.
M3	Error analysis notebook complete. Test-set metrics on final models documented.
M4	Containerized. Deployed to demo VM. Integration tests passing.

Week 6 — Demo Prep

Inter n	Friday EOD Deliverable
M1	Final feature engineering report (1-page) bundled with eval report.
M2	Final model documentation finalized.
M3	Formal Model Evaluation Report (PDF, 5-8 pages) delivered.
M4	Full E2E demo flow validated. Latency p95 <300ms documented.

11

Cross-Team Integration

11.1 Who We Wait On

From	What	When
Data Analysts (DA1)	Cleaned CSVs in shared location	End of Week 1
Web (W4, W5)	DB schema for candidates/jobs/applications	End of Week 1
Web (W5)	Proxy endpoint /api/score forwarding to us	Week 4

11.2 Who Waits On Us

Who	What
GenAI (G3)	Working /score endpoint by Week 3 — needed to build /explain.
GenAI (G3)	SHAP value format spec by Week 3 — needed for explanation generation.
Web (W1)	Working /score by Week 4 — to render score cards in profile drawer.
Web (W3)	Working /score by Week 4 — to display scores in copilot results.

12

Definition of Done

An ML deliverable is 'done' only when ALL of these are true:

- ✓ Code is in a .py module — not a notebook — and merged to main.
- ✓ Tests pass: feature pipeline tests, model loading tests, API integration tests.
- ✓ Reproducible: random seed set, fixed data splits, documented hyperparameters.
- ✓ Models versioned in models/ directory with manifest.json.
- ✓ Metrics meet acceptance targets (Section 4.2).
- ✓ API responds within latency budget (Section 9.4).
- ✓ Fallback to rule-based baseline tested and verified.
- ✓ M3's evaluation report includes the model in scope.

CRITICAL

Models are easy to overfit on synthetic data. If your test set R^2 is suspiciously high (>0.98), you probably have data leakage. Audit your train/val/test split: same (candidate, job) pair should NEVER appear in both train and test.

HireAI Copilot — Machine Learning Engineers Documentation

Built by 4 Junior Interns. Guided by metrics. Powered by gradient boosting.

Confidential · Version 1.0 · 6-Week Sprint